Kimi Delta Attention (KDA)

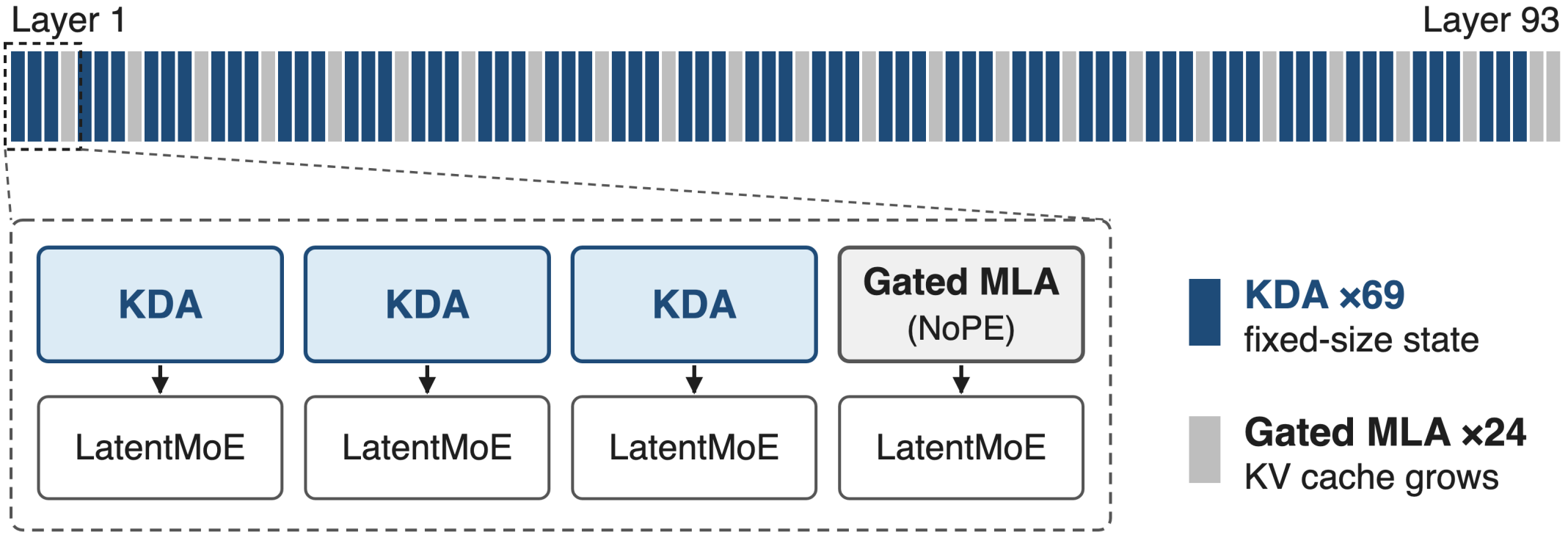
Delta-rule linear attention with per-channel forgetting

Moonshot AI · Kimi K3 (KDA introduced in Kimi Linear)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Attention & Long Context · arxiv.org/abs/2607.24653

Kimi K3 runs 69 of 93 attention layers on KDA



2.8T-parameter MoE · 104B active · 1M-token context window

Full attention pays for every past token

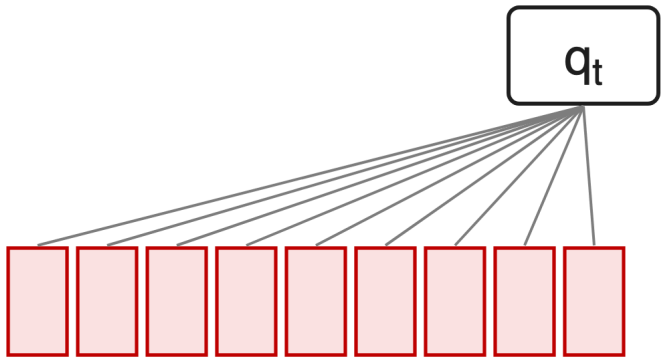
11.48 ms

per output token for a full-attention (MLA) model decoding at 1M context;
its KV cache grows with every token

Kimi Linear (arXiv 2510.26692) Fig. 1b: MLA baseline in the 48B-A3B setting (same layers and heads)

Growing caches and colliding writes both fail

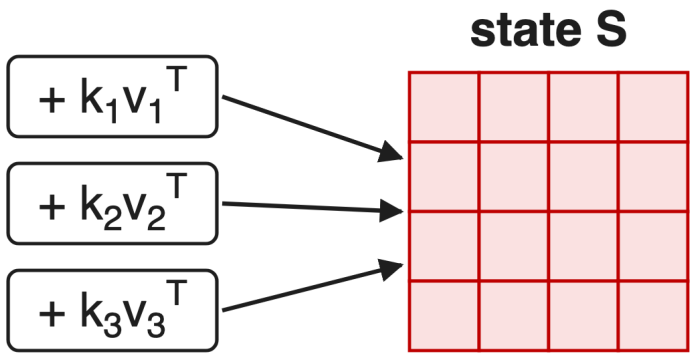
Softmax attention



KV cache: one entry per token

grows with every token

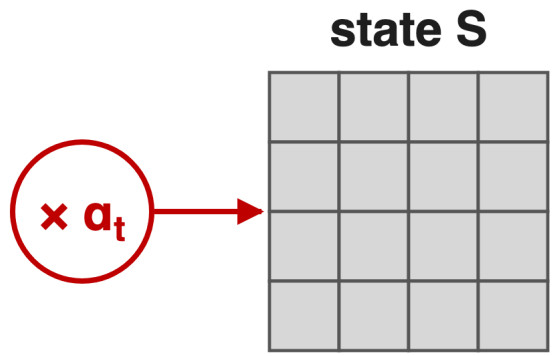
Linear attention



fixed $d_k \times d_v$ matrix

writes pile up without forgetting

Head-wise gate



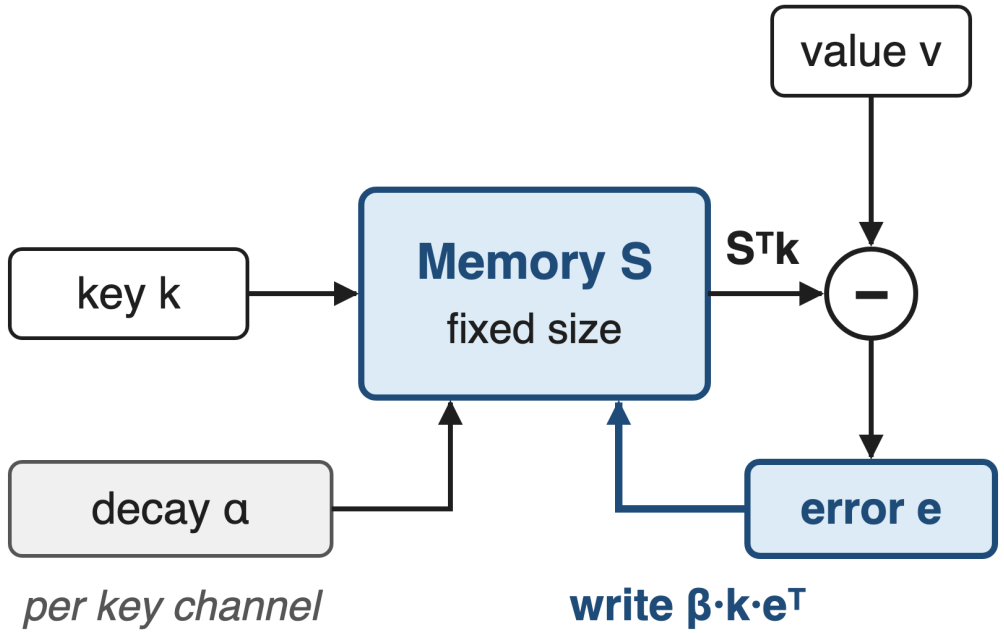
Mamba2 / GDN: one α per head

same decay for all features

With 128-dim keys at most 128 orthogonal keys fit: what to forget must be learned

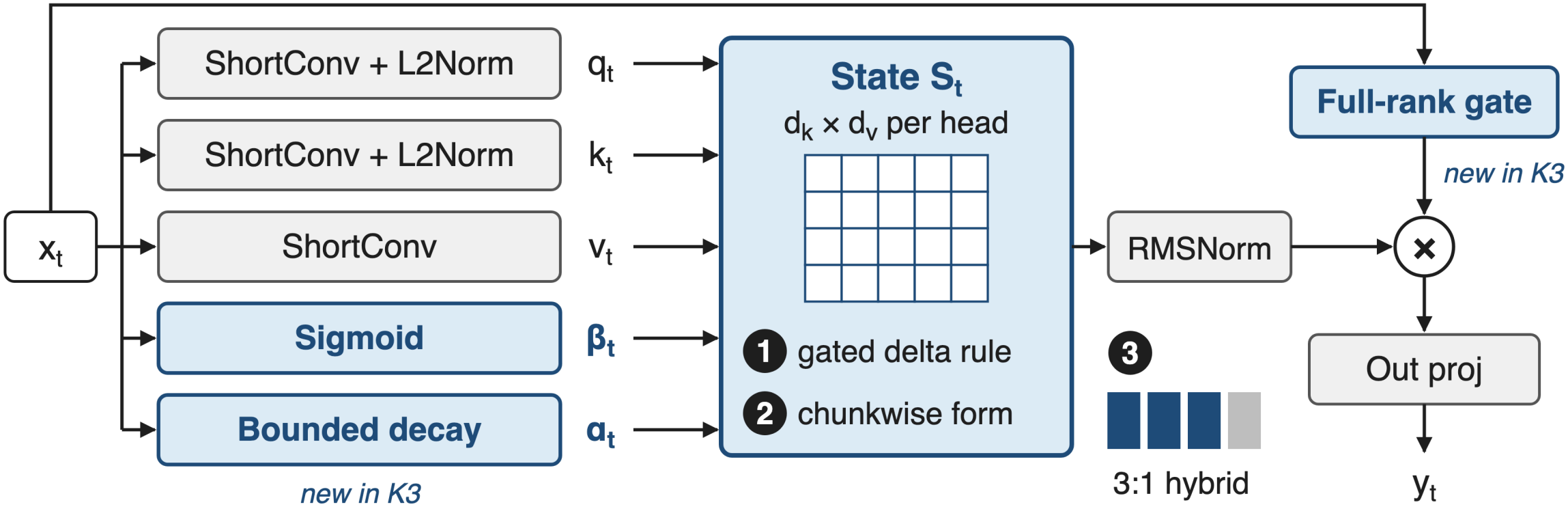
Insight: write only what memory gets wrong

- Delta rule = one gradient step on $\|S^T k - v\|^2$
- Decay each key channel, then write the error



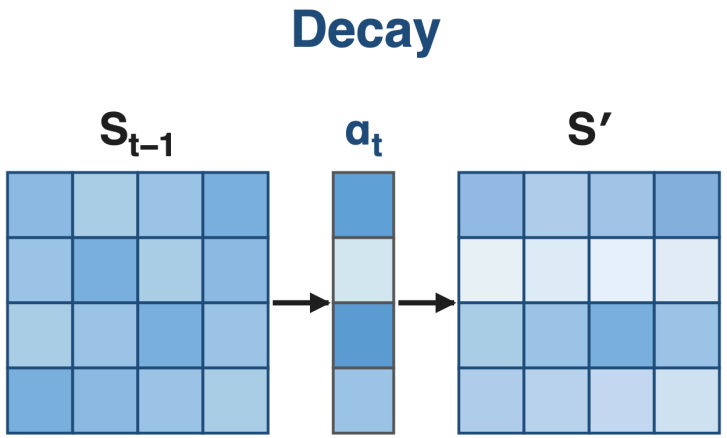
$S' = \mathrm{Diag}(\alpha) \cdot S$, then $S \leftarrow S' + \beta \cdot k \cdot (v - S'^T k)^T$: size never grows

One KDA layer: gated inputs, one fixed state



❶ state update · ❷ chunkwise execution · ❸ 3:1 hybrid with Gated MLA; blue = KDA-specific parts

❶ Each token: decay, predict, correct



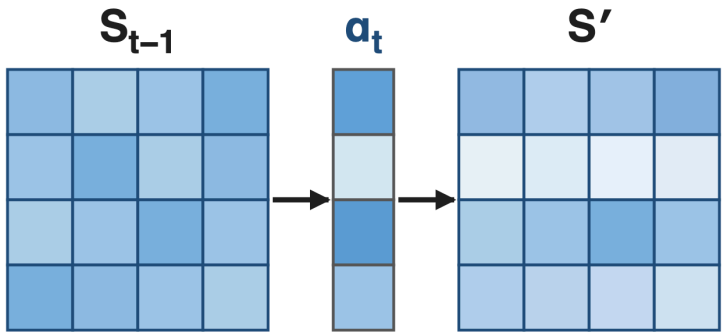
$S' = \mathrm{Diag}(\alpha_t)\, S_{t-1}$

forget per key channel

Same as K3 Eq. 1: new S = (I – β·k·k^T)·Diag(α)·old S + β·k·v^T

❶ Each token: decay, predict, correct

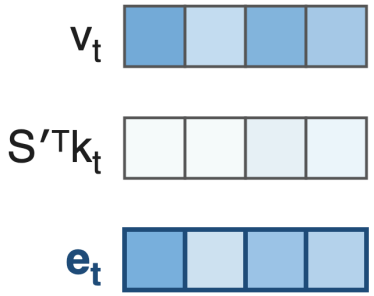
Decay



$$S' = \mathrm{Diag}(\alpha_t)\, S_{t-1}$$

forget per key channel

Predict



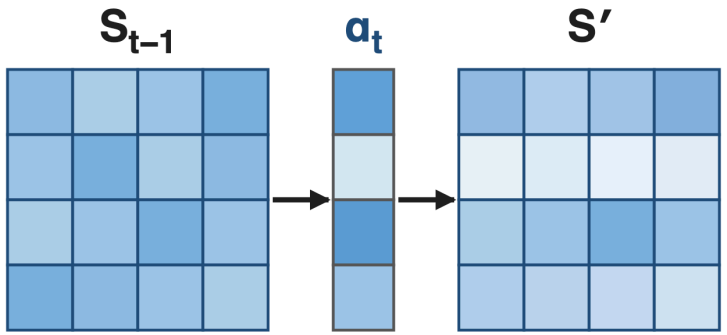
$$e_t = v_t - S'^T k_t$$

what memory gets wrong

Same as K3 Eq. 1: new S = (I – β·k·k^T)·Diag(α)·old S + β·k·v^T

❶ Each token: decay, predict, correct

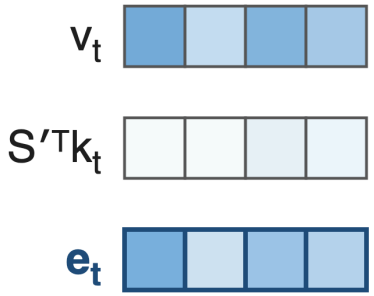
Decay



$$S' = \mathrm{Diag}(\alpha_t)\, S_{t-1}$$

forget per key channel

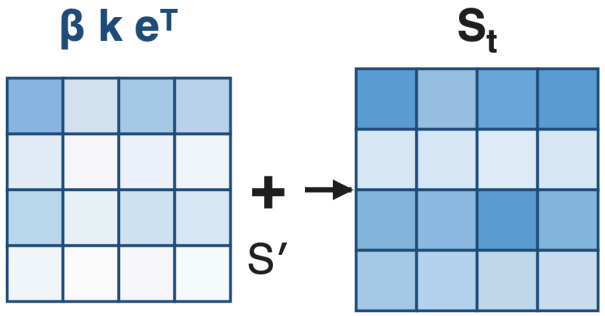
Predict



$$e_t = v_t - S'^T k_t$$

what memory gets wrong

Correct



$$S_t = S' + \beta_t\, k_t\, e_t^T$$

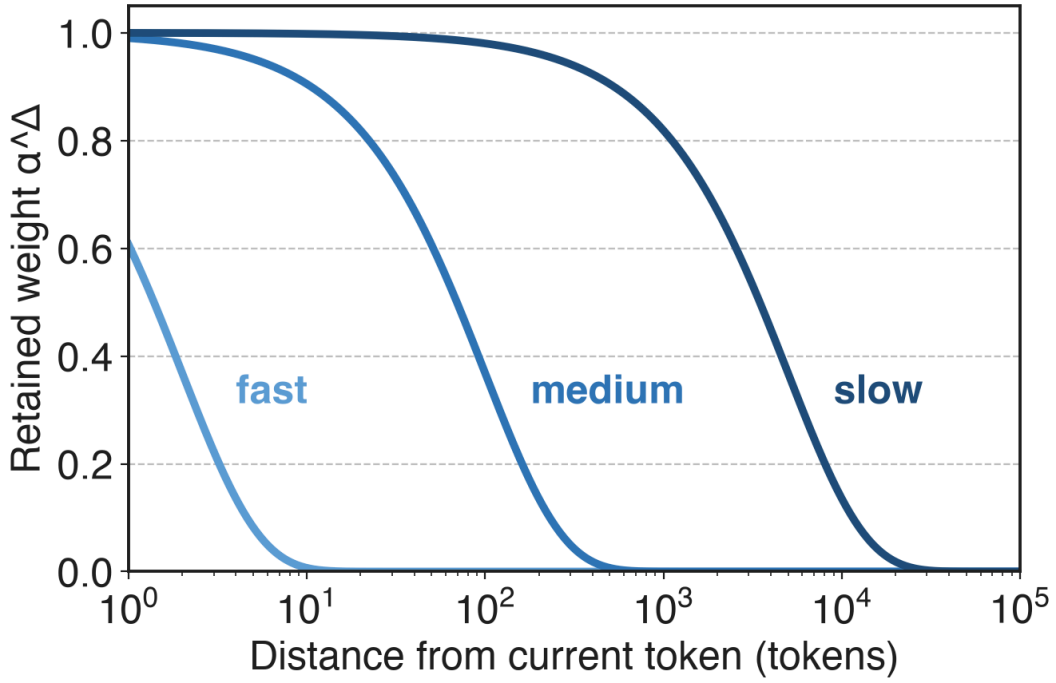
rank-1 write of the error

output $o_t = S_t^T q_t$ · state stays $d_k \times d_v$

Same as K3 Eq. 1: new S = (I – β·k·k^T)·Diag(α)·old S + β·k·v^T

❶ Per-channel decay also encodes position

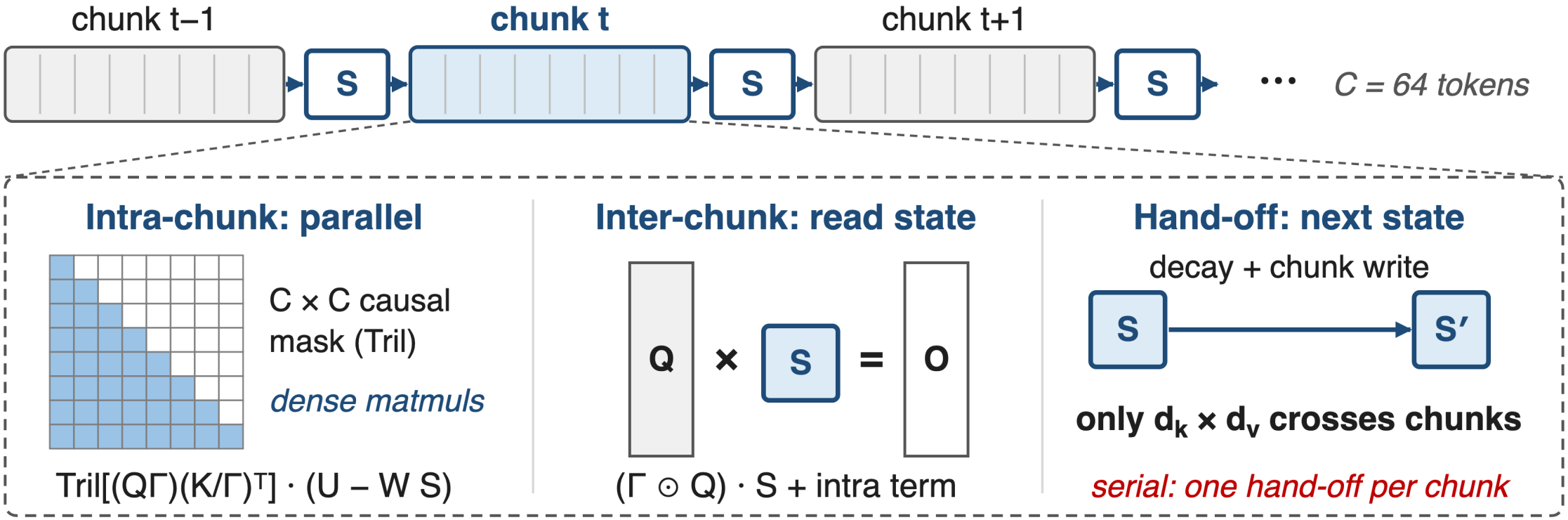
- One α per key channel, not per head
- Channels keep short to long horizons, like RoPE frequencies
- So all MLA layers drop RoPE (NoPE)



analytic, constant α per channel for illustration

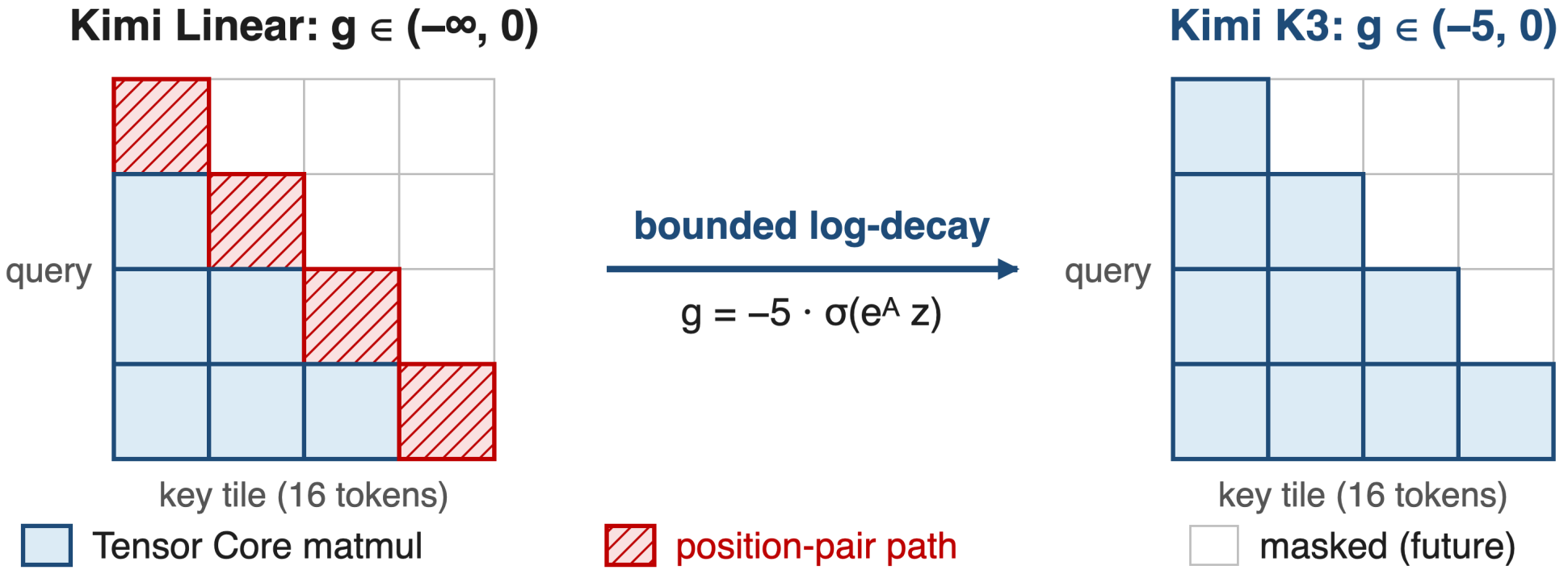
Synthetic recall and state tracking: KDA leads GDN at 256-2,048 tokens

❷ Chunks: parallel inside, recurrent across



Kernel that overlaps the two phases: FlashKDA, deck 08

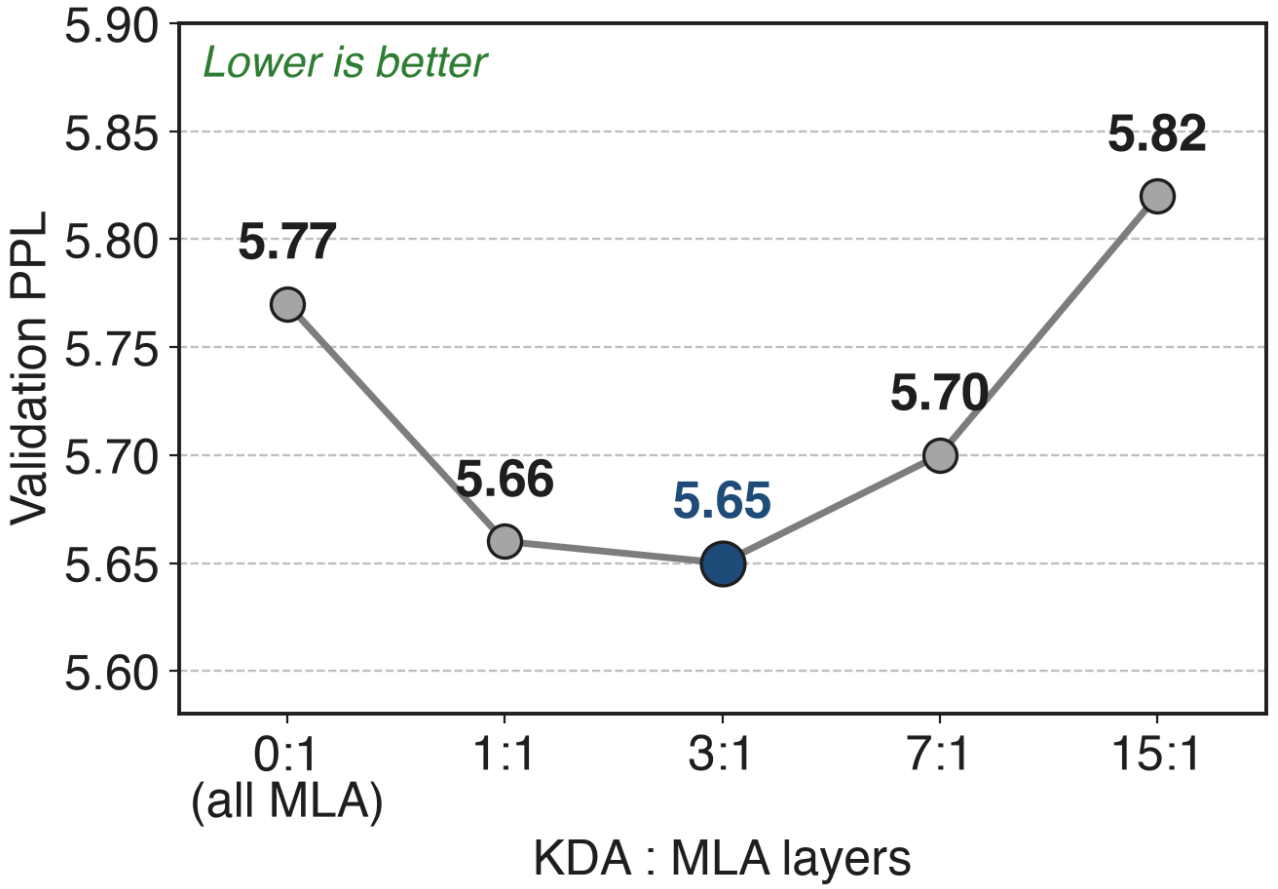
❷ Bounded decay: all tiles on Tensor Cores



Per-token g > –5, so over a 16-token tile $1/\Gamma < e^{80}$: inside BF16 range

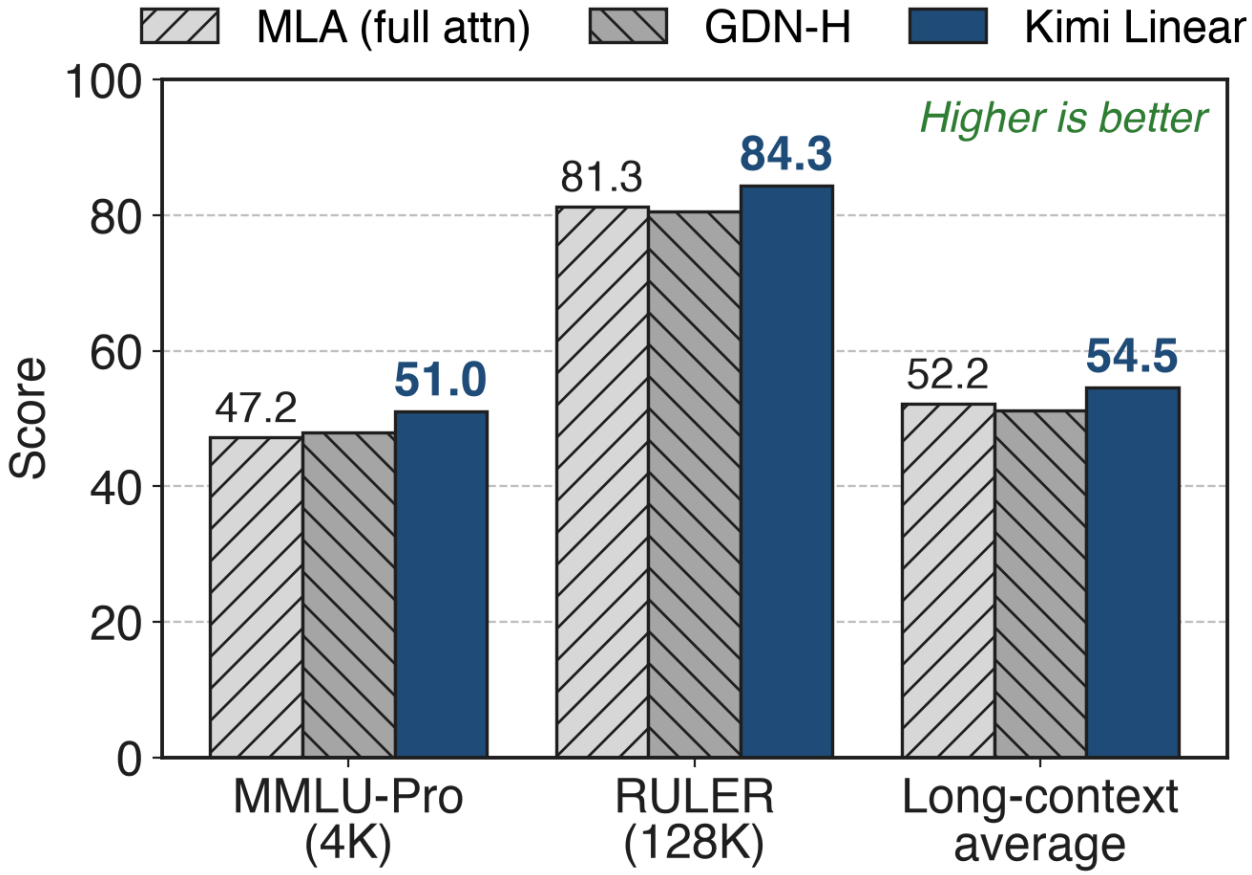
❸ Three KDA layers per global MLA layer

- A finite state cannot copy exactly: keep some global layers
- 3:1 is best; 7:1 and 15:1 lose on validation
- K3 adds one final MLA layer



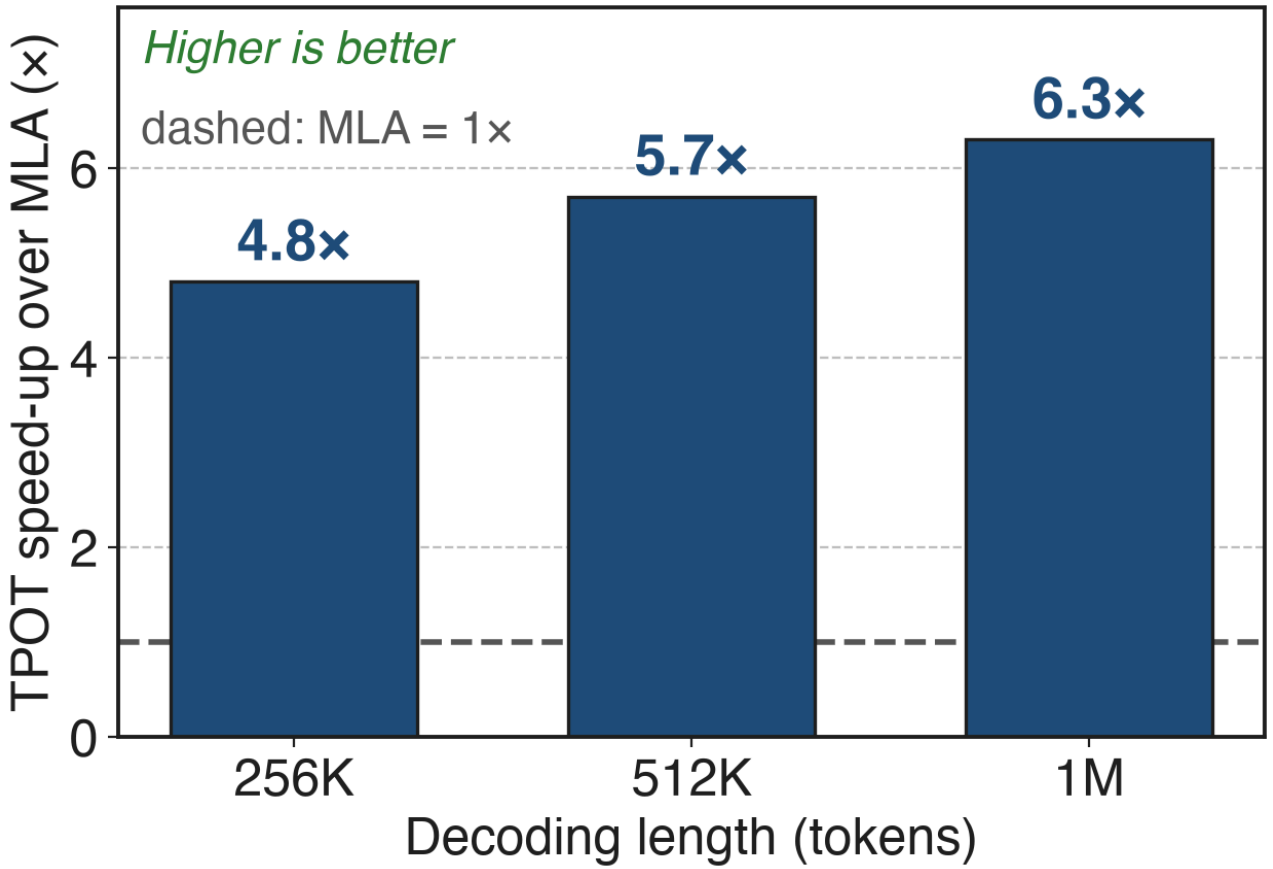
Same 1.4T tokens: KDA hybrid beats full MLA

- 48B-A3B MoE, identical data and recipe
- Wins short and long context
- Scaling-law fit: about **1.16×** compute efficiency over MLA



Decodes 6.3× faster than MLA at 1M tokens

- **1.84 ms** vs 11.48 ms per token at 1M
- Up to **75%** less KV cache allows larger batches
- Batch size 1: 2.2× decode, 2.9× prefill at 1M



At K3 scale: 2.5× scaling efficiency over K2

≈2.5×

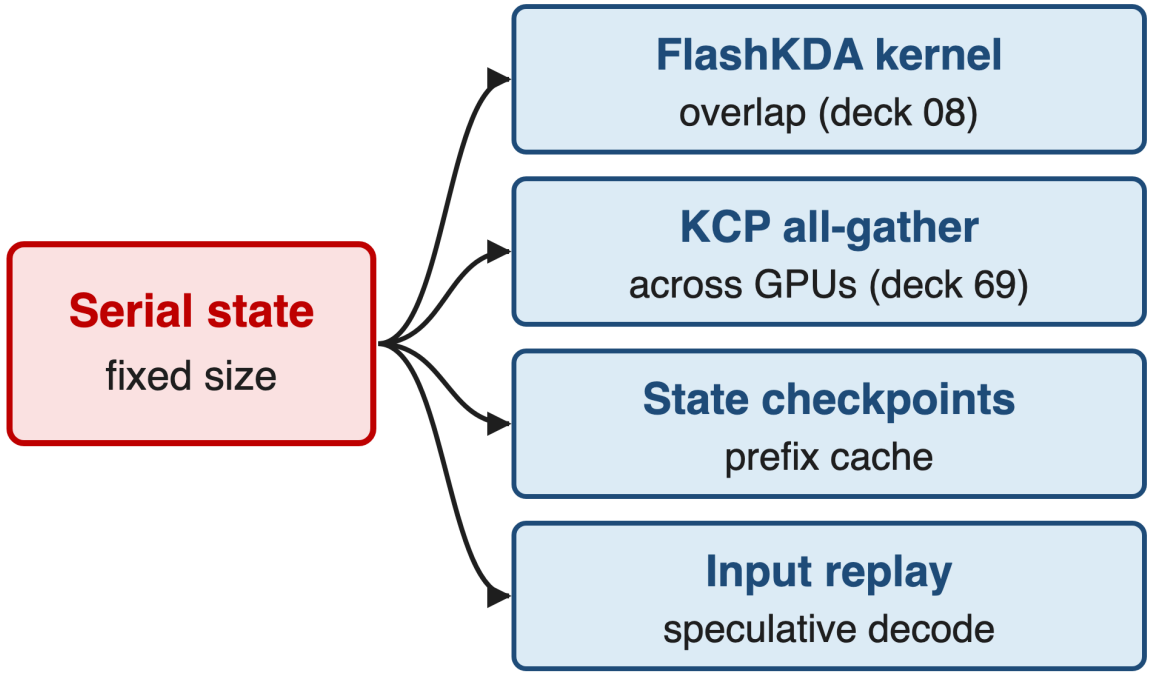
Kimi K3 vs K2 scaling efficiency, from KDA + AttnRes + Stable LatentMoE + data and training recipes together

- 1M-token context with no RoPE rescaling or interpolation

Kimi K3 report (arXiv 2607.24653): abstract, Sec. 3.2 and Fig. 7 (scaling laws), Sec. 3.4 (NoPE context extension)

Costs: long context only, new kernels and caches

- Gains need long context: prefill ≈ MLA at 4K-16K
- 24 MLA layers still keep a KV cache
- No KDA-only ablation at K3 scale



Summary

- Full attention at 1M: **11.48 ms** per token, cache grows with every token
- **KDA**: per-channel decay, then write only the error into a fixed state
- **6.3×** faster decoding at 1M; better quality at equal tokens
- K3: 69 of 93 layers; kernel in deck 08, CP in deck 69

https://arxiv.org/abs/2607.24653